

Programowanie geoprzetwarzania

Eksploracja danych przestrzennych



- Sprawdzamy, czy dane istnieją
- Jaki format mają?
- Tworzymy warstwy dostępnych warstw – podstawy iteracji po warstwach

Czy dane istnieją – funkcja **arcpy.Exists()** (na wyjściu: boolean)

Jest ważna, bo w Python nie ma dostępnych od razu narzędzi, które sprawdzają np. poprawność nazw warstw w skryptach

Ćwiczenie 1

Napisz skrypt – sprawdź, czy w geobazie (lub katalogu workspace) istnieje warstwa shapefile o nazwie roads92

'''

```
1  # importujemy moduł arcpy
2  import arcpy
3
4  # ustawiamy środowisko
5  #arcpy.env.workspace = r"Z:\DYDAKTYKA\rok_ak_2025_2026
6  arcpy.env.workspace = r"C:\Users\akolanek\Documents\Ar
7  arcpy.env.overwriteOutput = True
8
9  infc = "roads92"
10
11 print(arcpy.Exists(infc))
```

Sprawdzanie, czy istnieją dane (exists)

Dwa rodzaje katalogów:

- systemowe, rozpoznawane również przez Pythona, np. „C:/Data/warstwa.shp”

Obsługiwane także przez funkcje Python, np. `os.path.exists`

- Rozpoznawane TYLKO przez ArcGIS, np. „C:/Data/study.gdb/warstwa”

Mogą być obsługiwane przez arcpy (np. `Exists`)

Różne narzędzia geoprzetwarzania – różne dane

Typ danych i związane z nim właściwości sprawdza funkcja **Describe()**

Zwraca ona obiekt z właściwościami typu danych (np. geometria)

Sprawdź jaki typ geometrii ma warstwa roads92

Właściwości można wylistować poprzez: `print(arcpy.da.Describe(infc))`

Później odwołujemy się poprzez:

`arcpy.Describe(warstwa).właściwość`

```
1  # importujemy moduł arcpy
2  import arcpy
3
4  # ustawiamy środowisko
5  #arcpy.env.workspace = r"Z:\DYDAKTYKA\rok_ak_2025_2026\LETNI\PROJ"
6  arcpy.env.workspace = r"C:\Users\akolanek\Documents\ArcGIS\Proj"
7  arcpy.env.overwriteOutput = True
8
9  infc = "roads92"
10
11  #print(arcpy.da.Describe(infc))
12
13  desc = arcpy.Describe(infc)
14  print(desc.dataType)
15  print(desc.shapeType)
16
17  print(arcpy.Describe(infc).dataType)
```

Ćwiczenie 2

Napisz skrypt, który:

1. Ustawi workspace na geobazę

2. Zdefiniuje trzy zmienne:

infc - roads92

clipfc - wrocbuf

outfc - wroc_roads

3. Sprawdzi czy warstwa clipfc jest obiektem typu poligon

4. Jeżeli tak – dotnie do jej obszaru infc, a wynik zapisze jako outfc, dodatkowo poinformuje o tym w formie komunikatu wyświetlanego na ekranie (użyj print() i polecenia format())

Jeżeli nie – poinformuje o tym użytkownika (użyj print() i polecenia format())

5. Wykonaj to samo dla zmiennej clipfc, ale wcześniej przypisz jej warstwę rstations92


```
2 import arcpy
3
4 # ustawiamy środowisko
5 #arcpy.env.workspace = r"Z:\DYDAKTYKA\rok_ak_2025_2026\LETNI\PROGGEOP\DANE_STU
6 arcpy.env.workspace = r"C:\Users\akolanek\Documents\ArcGIS\Projects\proggeop20
7 arcpy.env.overwriteOutput = True
8
9 #zmienne
10 infc = "roads92"
11 clipfc = "wrocbuf"
12 outfc = "wroc_roads"
13
14 # przetwarzanie
15 desc = arcpy.Describe(clipfc)
16 |
17 if desc.shapeType == "Polygon":
18 |     arcpy.analysis.Clip(infc,clipfc,outfc)
19 |     print("Warstwa {0} została docięta do {1}!".format(infc,clipfc))
20 else:
21 |     print("Błąd, twój {0} nie jest poligonem!".format(clipfc))
```

Ćwiczenie 3

Napisz skrypt, który:

1. Sprawdzi, czy w geobazie istnieje warstwa nuts92
2. Jeśli tak, z pomocą Describe() sprawdzi i wypisze na ekranie:
 - Typ geometrii warstwy
 - Nazwę warstwy
 - Nazwę katalogu, w której warstwa się znajduje
 - Układ współrzędnych

```
#zmienne
szukana = "nuts92"

# print(arcpy.da.Describe(szukana))

czy_jest = arcpy.Exists(szukana)
if czy_jest is True:
    desc = arcpy.Describe(szukana) #możemy przypisywać zmienne wewnątrz instrukcji
    print("Nazwa warstwy {0}: {1}".format(szukana,desc.baseName))
    print("Ścieżka do warstwy {0}: {1}".format(szukana,desc.catalogPath))
    print("Typ geometrii warstwy {0}: {1}".format(szukana,desc.shapeType))
    print("Układ współrzędnych warstwy {0}: {1}".format(szukana,desc.spatialReference.name))
else:
    print("Warstwa nie istnieje...")
```

Lista umożliwia automatyzację skryptów:

- Powtarzanie operacji dla wszystkich warstw w katalogu
- Powtarzanie operacji dla warstw spełniających jakieś kryteria (określona geometria, układ współrzędnych itp.)

Po co lista? Do iterowania po nich (np. pętla for)

Jak tworzymy listy warstw z ArcPy:

- **ListFields**
- ListIndexes
- ListDatasets
- **ListFeatureClasses**
- **ListFiles**
- **ListRasters**
- **ListTables**
- ListWorkspaces
- ListVersions

Większość funkcji wymaga jedynie zdefiniowania workspace

Wszystkie domyślnie robią listę wszystkiego – wildcard * jako filtr

Filtr może być zmieniony

Przykładowa składnia:

```
ListFeatureClasses({wild_card}, {feature_type}, {feature_dataset})
```

Ograniczenie wyboru:

- po nazwie (wild_card)
- po typie obiektu (feature_type)
- po zestawie obiektów

Listy jako **obiekty** mają swoje metody, których możemy na nich użyć

Jedną z takich metod jest metoda `sort()`, wywoływana zgodnie z:

`obiekt.metoda(parametry)`

`obiekt.metoda` – kropka oznacza „użyć czegoś co należy do tego obiektu”

```
przykładowa_lista = [3, 1, 2]
```

```
przykładowa_lista.sort()
```

`sort()`:

działa tylko dla list,

modyfikuje listę na miejscu

2 parametry: `key` i `reverse` – jedno definiuje po czym będzie sortowanie; druga ma wartości logiczne

`True/False`

Ćwiczenie 4

Napisz skrypt, który:

1. Stworzy listę warstw typu `FeatureClasses` z geobazy - skorzystaj z `ListFeatureClasses()` i jej domyślnej właściwości
2. Wyświetli na ekranie jej elementy
3. Korzystając z pętli `for` wypisze na ekranie nazwy wszystkich warstw z geobazy, w odwrotnym porządku (od z do a) – użyj polecenia `sort()` z odpowiednim parametrem

lista.py / ...

```
1  # importujemy moduł arcpy
2  import arcpy
3
4  # ustawiamy środowisko
5  #arcpy.env.workspace = r"Z:\DYDAKTYKA\rok_al
6  arcpy.env.workspace = r"C:\Users\akolanek\Di
7  arcpy.env.overwriteOutput = True
8
9  lista = arcpy.ListFeatureClasses()
10 print(lista)
11 lista.sort(reverse=True)
12 for x in lista:
13     print(x)
```


Ćwiczenie 5

Zrób kopię katalogu raster

Napisz skrypt, który:

Dla każdej warstwy rastrowej z tego katalogu zbuduje piramidy

Skorzystaj z:

`ListRasters()`, `management.BuildPyramids()`

Sprawdź rezultat

```
1  # importujemy moduł arcpy
2  import arcpy
3
4  # ustawiamy środowisko
5  arcpy.env.workspace = r"Z:\DYDAKTYKA\rok_ak_2025_2026\LETNI\PROGGEOP\raster"
6  #arcpy.env.workspace = r"C:\Users\akolanek\Documents\ArcGIS\Projects\proggeop2026
7  arcpy.env.overwriteOutput = True
8
9  lista = arcpy.ListRasters()
10 print(lista)
11 for x in lista:
12     arcpy.BuildPyramids_management(x)
```

Przydatne np. gdy chcemy przyciąć wszystkie sceny satelitarne do wybranego obszaru

Ćwiczenie 6

Sprawdź, jakie pola znajdują się w tabeli atrybutów warstwy nuts92 - skorzystaj z ListFields()

Wyświetl:

- Wszystkie pola
- Wydrukuj nazwę i typ pola na ekranie (metody: name, type)

```

9   warstwa = "nuts92"
10  lista = arcpy.ListFields(warstwa)
11  for i in lista:
12      print("Kolumna {0} ma typ: {1}".format(i.name,i.type))
13
14

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

- PS Z:\DYDAKTYKA\rok_ak_2025_2026\LETNI\PROGGEOP\skrypty> & "C:/Program Files/ArcGIS, AKTYKA/rok_ak_2025_2026/LETNI/PROGGEOP/skrypty/lista.py
 Kolumna OBJECTID ma typ: OID
 Kolumna Shape ma typ: Geometry
 Kolumna IDS ma typ: String
 Kolumna NAZWA ma typ: String
 Kolumna RODZAJ ma typ: String
 Kolumna ID_WOJ ma typ: String
 Kolumna ID_POWIAT ma typ: String
 Kolumna ID_GMINA ma typ: String
 Kolumna ID_RODZAJ ma typ: String
 Kolumna ID_WYR ma typ: String
 Kolumna ID_49_WOJ ma typ: String
 Kolumna ID_49_GMIN ma typ: String
 Kolumna IDS_49_GMI ma typ: String
 Kolumna SUB ma typ: SmallInteger
 Kolumna Shape_Length ma typ: Double
 Kolumna Shape_Area ma typ: Double
- PS Z:\DYDAKTYKA\rok_ak_2025_2026\LETNI\PROGGEOP\skrypty> █

Ćwiczenie 7

Zmodyfikuj skrypt tak, żeby wyświetlił nazwy tylko pól typu String

(istnieją co najmniej dwa sposoby, oparte o rozwiązania przedstawiane na zajęciach)

```
warstwa = "nuts92"
lista = arcpy.ListFields(warstwa)
for i in lista:
    if i.type == "String":
        print(i.name)
```

```
warstwa = "nuts92"
lista = arcpy.ListFields(warstwa, field_type="String")
for i in lista:
    print(i.name)
```

Zadanie 1:

Napisz skrypt, który:

Dla wszystkich warstw z geobazy, zawierających obiekty poligonowe dotnie wszystkie obiekty liniowe

Nazwy wynikowych warstw zdefiniuj według schematu:

linia_poligon – gdzie poligon to nazwa warstwy docinającej, a linia – docinanej

Zadanie 2.

Napisz skrypt, który usunie wszystkie przygotowane w poprzednim zadaniu warstwy i TYLKO TE warstwy.

Do kasowania można użyć Delete()

UWAGA.

Lepiej pracować na kopii geobazy lub przed wykonaniem operacji usuwania zakomentować tę linię i zrobić sprawdzenie (np. wyprintować nazwy warstw do usunięcia w iteracjach)

Zadanie 3.

Napisz skrypt, który:

dla wszystkich warstw z geobazy, zawierających obiekty poligonowe

- Dotnie wszystkie warstwy liniowe

Nazwy wynikowych warstw zdefiniuj według schematu: clip_linia_poligon

(poligon - warstwa docinająca, linia – warstwa docinana)

- Sprawdzi, czy w wyniku docinania nie powstała warstwa pusta.

Jeśli tak:

Napisze komunikat na ekranie

Usunie od razu warstwę

Jeśli nie:

Napisze komunikat na ekranie